



Engineering Analytics
INDE 6360
Fall 2025

Cullen College of Engineering
University of Houston

Job Recommendation System Based on Skills

Team:
Abrar Hamim Masih
Anish Khanal
Shail Shah
Yatrikkumar Shah

Submission Date: December 12, 2025

Table of Contents

Abstract.....	1
1. Introduction.....	2
1.1 Background.....	2
1.2 Motivation.....	2
1.3 Objectives	2
2. Problem Statement	3
2.1 Job Seeker Challenges	3
2.2 Recruiter Challenges	3
2.3 Technical Limitations of Existing Systems	3
2.4 Lack of standardization.....	3
3. Proposed Solution	4
3.1 Skill Extraction	4
3.2 Semantic Embedding Generation.....	4
3.3 Skill Normalization through Ontology Mapping	4
3.4 Similarity Computation.....	4
3.5 Actionable Recommendations	4
4. System Architecture.....	5
4.1 Architecture Overview.....	5
4.2 Component Description	5
4.2.1 Resume Parsing.....	5
4.2.2 Job Description Processing	6
4.2.3 Skill Extraction	6
4.2.4 Skill Embedding Generation.....	6
4.2.5 Skill Normalization and Ontology Mapping.....	7
5. Methodology	8
5.1 Resume Parsing Flow	8
5.1.1 Getting Documents In	8
5.1.2 Cleaning and Preparing.....	8
5.1.3 Identifying the Section.....	8
5.2 Flow for Parsing Job Descriptions	9
5.2.1 JDs Ingestion.....	9
5.2.2 Finding the Section for JDs.....	9
5.3 Normalisation of Skills and Mapping of Ontologies	9
5.3.1 The Problem of Normalisation.....	10

5.3.3 Advantages of Normalisation.....	10
5.4 Vector Encoding and Embeddings.....	10
5.4.1 The Old Way: TF-IDF Encoding	11
5.4.2 Our Method: Transformer Embeddings.....	11
5.5 Recommendations and Rankings	11
5.5.1 Finding Similarities.....	12
5.5.2 Scoring in More Than One Dimension	12
5.5.3 Suggestion Output.....	12
6. Model Used.....	13
6.1 Qwen (Extensive Linguistic Model)	13
6.2 GLiNER (Named Entity Recognition Model)	13
6.3 Sentence Transformers (Embedding Model)	13
7. Design Rationale: Unsupervised Learning	14
7.1 No Labelled Datasets	14
7.2 Pretrained Models Encode Knowledge	14
7.3 Natural Fit for Similarity-Based Matching	14
7.4 Maintainability and Adaptability	15
8. Implementation Details.....	15
8.1 Stack.....	15
8.2 System Requirements.....	15
8.3 Code Structure	15
9. Application Demo and Results	16
9.1 User Interface.....	16
9.2 Sample Output	16
10. Future Scope	17
10.1 Model Improvement.....	17
10.2 User Feedback Integration	17
10.3 Enhanced User Interface	17
10.4 Advanced Filtering and Business Logic	17
Conclusion	18
References.....	19

Abstract

The current job market is characterized by an excessive number of job postings and resumes, making it increasingly difficult for job applicants to secure appropriate positions and for recruiters to identify qualified candidates. Traditional keyword-based matching systems are unable to accurately understand the semantic significance of skills and experiences, resulting in less effective matches. This project introduces a Job Recommendation System that employs Natural Language Processing (NLP), Large Language Models (LLMs), and semantic embeddings to deliver intelligent job matching. Our system identifies skills from resumes and job descriptions through a hybrid methodology that integrates LLM-based extraction (Qwen) with Named Entity Recognition (GLiNER), followed by normalisation of the extracted skills through an ontology-based mapping system to address variations, synonyms, and abbreviations. By transforming documents into dense semantic vectors using Sentence Transformers and computing cosine similarity, the system effectively captures genuine semantic alignment rather than relying solely on keyword overlap. The unsupervised methodology employed in our approach eliminates the need for costly labelled datasets while maintaining high accuracy through zero-shot semantic similarity. The system provides actionable recommendations, including talent matching, identification of skill gaps, and confidence scoring, thereby enhancing efficiency for both job seekers and recruiters.

Keywords: Job Recommendation, NLP, Large Language Models, Semantic Embeddings, Named Entity Recognition, Skill Extraction, Ontology Mapping, Sentence Transformers, Unsupervised Learning

1. Introduction

1.1 Background

The recruiting sector has experienced a substantial transformation due to the digitisation of job advertisements and CV submissions. Online job platforms such as LinkedIn, Indeed, Glassdoor, and Monster have facilitated job applications, although this convenience has introduced a range of issues. Recruiters currently receive hundreds or even thousands of applications for a single position, while job searchers find it challenging to discern whether possibilities genuinely align with their skill sets. This project used a dataset obtained from Kaggle, entitled Job Descriptions 2025 – Tech and Non-Tech Roles [1], comprising around 1,100 job descriptions across 55 unique positions, detailing skills, responsibilities, and experience levels.

The conventional method of job matching predominantly depends on keyword-based searches and filters. When a recruiter advertises a position requiring "Python" expertise, the system scans the resume for that specific term. This method overlooks candidates who may have included "Python programming," "Python3," or "Py" on their resumes. An applicant seeking "Data Science" positions may overlook roles designated as "Data Analyst" or "ML Engineer," even when they possess the necessary qualifications.

1.2 Motivation

The boundaries of keyword-based matching provide considerable real-world consequences. Job seekers invest extensive time applying for positions that do not genuinely align with their qualifications, while competent candidates are overlooked due to mismatches in language. Recruiters spend time evaluating non-qualified applications, resulting in prolonged hiring processes and increased costs.

Recent advancements in Natural Language Processing and Large Language Models have created new opportunities for sophisticated text comprehension. Models such as BERT, GPT, and their variants can comprehend context, synonyms, and semantic relationships in ways that go beyond keyword matching. The study focuses on using these advancements to develop a more intelligent job suggestion system.

1.3 Objectives

- To develop an intelligent system that utilises NLP and LLMs to extract skills from resumes and job descriptions.
- To execute skill normalisation through ontology-based mapping to address variances and synonyms.
- To produce semantic embeddings that accurately reflect the underlying meaning of resumes and job descriptions.
- To compute significant similarity metrics between candidates and job listings.
- To provide actionable recommendations, including matching skills, skill gaps, and confidence scores.

2. Problem Statement

The existing job matching environment encounters multiple interrelated issues that our system is working to resolve:

2.1 Job Seeker Challenges

Job seekers frequently have difficulty recognising positions that genuinely align with their competencies. The extensive array of job postings across various platforms renders it virtually infeasible to manually assess all potentially suitable positions. Moreover, discrepancies in wording between candidates' resumes and recruiters' job descriptions result in wasted chances. A software developer proficient in "RESTful APIs" may not be found in searches for "REST web services," despite the abilities being equivalent.

2.2 Recruiter Challenges

Recruiters receive numerous resumes for each position and find it challenging to filter candidates efficiently. Manual screening is labour-intensive, costly, and susceptible to human bias. The absence of standardisation in candidates' presentation of qualifications renders automated filtering problematic, frequently leading to the rejection of eligible candidates while allowing less appropriate ones to progress.

2.3 Technical Limitations of Existing Systems

Current job-matching services, including prominent platforms such as LinkedIn, predominantly utilise keyword-based matching. This methodology possesses fundamental limitations:

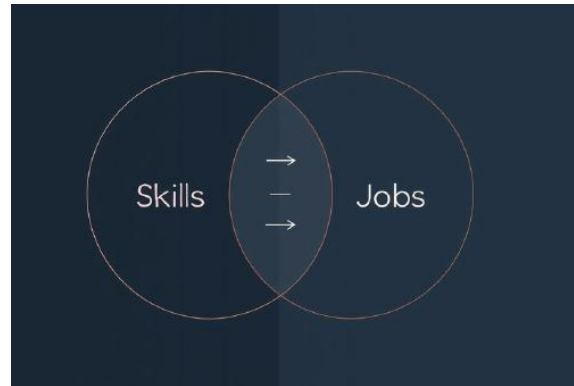
- **Semantic blindness:** Keyword matching fails to recognise that "ML" and "Machine Learning" denote the same concept.
- **Contextual confusion:** The term "Python" may denote either a programming language or a serpent, i.e., keyword systems are unable to differentiate.
- **Skill distinctions:** "Deep learning," "DL," and "Neural networks" are interconnected yet regarded as entirely separate concepts.
- **Absence of implicit skill inference:** While proficiency in TensorFlow typically implies familiarity with Python, keyword systems are incapable of making such deductions.

2.4 Lack of standardization

There is no one way to write a CV or a job description. There are several ways that candidates style their resumes. They distribute their talents across different parts, use other words, and organise their material in various ways. Job descriptions also differ substantially in structure and wording across companies and industries. Because there is no standardisation, it is tough to make matches that are always the same.

3. Proposed Solution

Our Job Recommendation System tackles the identified challenges through a multi-stage pipeline that employs advanced NLP techniques. The solution consists of five essential elements:



3.1 Skill Extraction

We utilise a hybrid methodology that integrates Large Language Models (LLMs) with Named Entity Recognition (NER) to extract skills from resumes and job descriptions. The LLM component comprehends context and is capable of recognising implicit skills, whereas the NER component delivers precise extraction of explicit technical entities.

3.2 Semantic Embedding Generation

Instead of viewing documents as collections of keywords, we transform complete resumes and job descriptions into dense semantic vectors utilising transformer-based sentence embeddings. These embeddings encapsulate the semantic meaning and contextual information of the entire document, facilitating similarity assessments that extend beyond superficial text matching.

3.3 Skill Normalization through Ontology Mapping

Extracted skills are standardised using an ontology-based system that maps variations, synonyms, and abbreviations to their canonical representations. For instance, "Deep learning," "DL," and "Neural nets" all correspond to the standardised entry "Deep Learning." This normalisation guarantees uniformity in comparisons across documents.

3.4 Similarity Computation

Our system evaluates similarity across various dimensions: skills-to-requirements correspondence and experience-to-responsibilities alignment. By integrating these scores with appropriate weights, we generate an exhaustive match score that accurately reflects the candidate's suitability for the position.

3.5 Actionable Recommendations

The concluding output offers job seekers and recruiters with practical insights, including:

- Ranked list of suggested jobs or candidates
- Skill alignment between resume and job requirements
- Identification of missing skills (gap analysis)

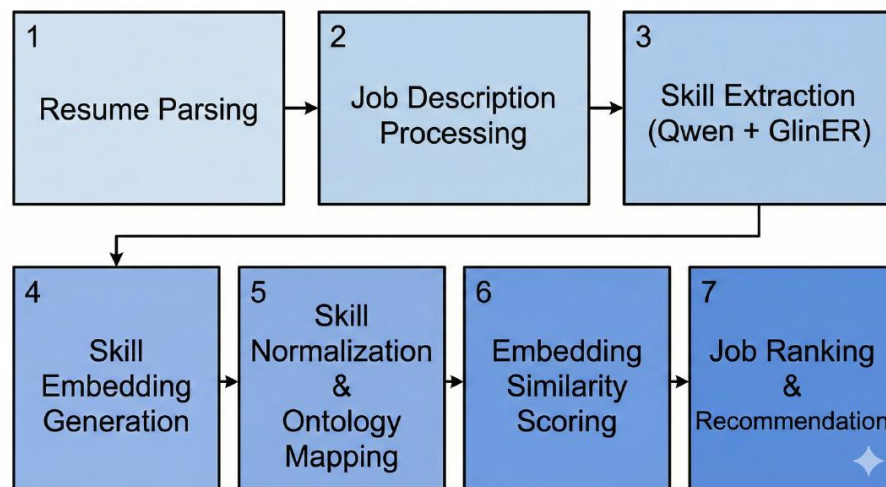
- Confidence scores assigned to each recommendation
- Indicators of profile match strength

4. System Architecture

The Job Recommendation System has a modular architecture that separates components and enables them to be scaled and updated independently. The architecture is based on a pipeline pattern, in which data flows through a series of processing steps.

4.1 Architecture Overview

There are three basic layers in the system: the Input Layer receives documents from different sources, the Processing Layer performs extraction, normalisation, and embedding, and the Output Layer identifies commonalities and generates suggestions.



4.2 Component Description

4.2.1 Resume Parsing

The Resume Parsing component functions as the initial interface for candidate data entry into the system. It accepts resumes in PDF format, the most prevalent format for professional documents. The component employs PDF parsing libraries such as PyPDF2 and pdfplumber to remove unprocessed text while preserving the logical reading sequence. This component not only extracts text but also performs preprocessing tasks, including whitespace normalisation, punctuation standardisation, lowercase conversion, and removal of non-UTF characters. The processed and refined text is thereafter transmitted to downstream components for additional analysis. This component supports diverse resume formats, including single-column, multi-column, and visually complex designs.

4.2.2 Job Description Processing

The Job Description Processing component manages the acquisition and organisation of job posts from various sources. It accommodates job descriptions sourced from LinkedIn, Indeed, corporate career pages, or manual text entry. This component, akin to resume parsing, sanitises and standardises the text to guarantee uniformity. Nevertheless, it performs JD-specific processing by identifying and delineating essential elements, including Requirements (required credentials) and Responsibilities (job duties). This structural comprehension is crucial as requirements delineate the necessary skills, whereas responsibilities articulate the actual tasks—both offer distinct alignment indicators. The job descriptions are recorded in a standardised manner for comparison with candidate profiles.

4.2.3 Skill Extraction

The Skill Extraction component identifies and extracts skills from parsed resumes and processed job descriptions. This component utilises a hybrid extraction strategy that integrates two complementing methodologies. The initial method employs Qwen, a large language model, adept at understanding contextual nuances in words, performing reasoning, and discerning both explicit skills (clearly articulated) and implicit abilities (derived from project descriptions or experiences). The second technique employs GLiNER, a Named Entity Recognition model that offers token-level accuracy in identifying domain-specific technical items with minimal hallucination. The system achieves broad coverage and accuracy by integrating LLM-based contextual comprehension with NER-based exact entity recognition.

4.2.4 Skill Embedding Generation

The Skill Embedding Generation component converts retrieved skills and document material into dense numerical vectors utilising Sentence Transformer models. This component generates compact semantic embeddings (usually 384-768 dimensions) that encapsulate the meaning and context of the text, in contrast to traditional TF-IDF encoding, which produces sparse, high-dimensional vectors based on word frequency. These embeddings are trained on large-scale text corpora and capture semantic relationships; for instance, "Machine Learning" and "ML" will have similar vector representations despite their textual dissimilarity. The component produces embeddings for specific talents, skill sets, and complete documents, facilitating multi-tiered similarity assessments in further processing.

4.2.5 Skill Normalization and Ontology Mapping

The Skill Normalisation and Ontology Mapping component addresses skill discrepancies, synonyms, and abbreviations in the extracted skill lists. Raw extraction frequently yields duplicates and inconsistent nomenclature; for instance, "Deep learning," "DL," and "Neural nets" may all be represented as distinct skills. This component aligns all extracted talents to a canonical skill dictionary (skill ontology) that includes standardised skill names, their recognised variations, and hierarchical categorisation. The mapping procedure employs embedding similarity measures to align variations with their canonical forms. If the embedding of an extracted term closely resembles that of an ontology entry, they are deemed matches. This normalisation rectifies naming discrepancies, consolidates synonyms into singular entries, eliminates duplicates, and facilitates superior semantic job matching.

4.2.6 Embedding Similarity Scoring, Job Ranking and Recommendation

The scoring component serves as the primary matching engine, evaluating the alignment between a candidate's profile and job requirements. It computes the cosine similarity between the resume embeddings and the job description embeddings, which quantifies the angular relationship between vectors in the embedding space, with values ranging from -1 (entirely dissimilar) to 1 (identical). The component performs multidimensional scoring by independently calculating skills-to-requirements and experience-to-responsibilities similarity, and then combining these scores with appropriate weights. Occupations are prioritised based on their aggregate similarity scores, and the leading N recommendations are chosen. The output comprises ranked jobs, corresponding skills, missing skills (gap analysis), similarity confidence scores, and profile match strength indicators, thereby making the data actionable for both job seekers and recruiters.

5. Methodology

This section explains the technical steps used at each stage of the job suggestion pipeline.

5.1 Resume Parsing Flow

The resume parsing pipeline transforms raw PDF documents into structured, analyzable data through several processing stages.

5.1.1 Getting Documents In

Resumes are submitted as PDFs, the most common format. To extract raw text from a PDF file while preserving the logical reading order, we use PDF parsing tools such as PyPDF2 or pdfplumber. This phase supports several types of PDF structures, including single-column and multi-column layouts, as well as layouts that are difficult to read.

5.1.2 Cleaning and Preparing

The text that was taken out goes through several cleaning steps operations using NLP libraries such as as spaCy [2] and NLTK:

- Normalisation of whitespace: Multiple spaces, tabs, and uneven spacing are all made the same.
- Handling of punctuation: Special characters and strange punctuation are all made the same.
- Case normalisation: The text is changed to lowercase so that it can be processed in the same way every time.
- Character encoding: Characters that aren't UTF are taken out or replaced.

5.1.3 Identifying the Section

Most resumes include separate parts for Skills, Experience, Education, Projects, and Certifications. Our algorithm identifies these parts so it can obtain the correct information in context. Two different but helpful methods are used:

Extraction Based on LLM:

- Uses reasoning to understand the context of texts.
- Gets both explicit skills (like "Proficient in Python") and implicit skills (like those that can be deduced from project descriptions).
- Effectively deals with resumes that are unclear or poorly organized.

NER-Based Extraction:

- Uses supervised Named Entity Recognition models.
- Finds domain-specific skill entities with high accuracy.
- Finds standard resume sections with high accuracy.

5.2 Flow for Parsing Job Descriptions

Parsing job descriptions is similar to parsing resumes, but it requires several modifications to account for the structure of job descriptions.

5.2.1 JDs Ingestion

You can obtain job descriptions from several sources, such as directly connecting to LinkedIn and Indeed via an API, scraping job sites, or typing them in by hand. The system accepts only manual input and converts it into a standard format for processing.

5.2.2 Finding the Section for JDs

Job descriptions are examined for Requirements (qualifications required) and Responsibilities (duties of the role), whereas resumes focus on Skills and Experience. These parts give several kinds of matching signals:

Extraction for JDs Based on LLM:

- Better at understanding complicated requirements.
- Can tell the difference between "nice to have" and "required" language.
- Follows the instructions given.
- The amount of time it takes to process relies on the resources of the system.

Extraction Based on GLiNER NER:

- Much faster than LLM-based extraction.
- Uses embedding-based token-level classification.
- Offers hybrid NER with contextual understanding.
- Only works for skills that are explicitly mentioned (can't figure out implicit needs).

5.3 Normalisation of Skills and Mapping of Ontologies

When you extract raw skills, you get lists with duplicates, variations, shorthand notations, and synonyms. If there were no normalisation, "Deep learning," "DL," and "Neural nets" would be

regarded as three distinct skills with no commonality. Our ontology mapping integrates these into a single set of standard terminology.

5.3.1 The Problem of Normalisation

A look at these examples of different skill levels:

Raw Variations	Canonical Form	Category
Deep learning, DL, Neural nets	Deep Learning	Machine Learning
Pandas, pandas library, Python dataframes	Pandas	Data Handling
JS, JavaScript, ECMAScript	JavaScript	Programming Languages

5.3.2 Structure of the Ontology

All removed skills are consistent with a standard skill dictionary (skill ontology). This dictionary lists conventional skill names, their most common variations, and a method for grouping them by level of importance. Embedding similarity measures are used to determine whether two terms match. If the embedding of an extracted phrase is close enough to that of an ontology entry, they are considered matches.

5.3.3 Advantages of Normalisation

The process of normalisation has several essential benefits:

- Consistency: Fixes problems with names that are different in different documents
- Deduplication: Combines synonyms into one entry, getting rid of duplicates.
- Better matching: Allows for high-quality semantic job matching.
- Scalability: New skills can be added to the ontology without changing the system.

5.4 Vector Encoding and Embeddings

To computationally compare resumes and job descriptions, we need to convert text into numeric vectors. This change is necessary to get valid similarity scores.

5.4.1 The Old Way: TF-IDF Encoding

The standard way to vectorise text is to use Term Frequency-Inverse Document Frequency (TF-IDF). It represents documents as word-frequency vectors, with each dimension corresponding to a word in the lexicon. Earlier embedding methods, such as Word2Vec[3], improved upon TF-IDF by learning distributed word representations but still had limitations in capturing sentence-level semantics.

TF-IDF has some problems:

- Makes sparse, high-dimensional vectors.
- Doesn't catch synonyms or semantic meaning.
- Treats words as separate units without context.
- Can't tell that "ML" and "Machine Learning" are connected.

5.4.2 Our Method: Transformer Embeddings

Our system uses pretrained Sentence Transformer models to produce dense semantic vectors [4]. These embeddings, on the other hand, capture the content and context of entire manuscripts, rather than merely word frequencies, as TF-IDF does. The Transformer architecture [5], first described in the publication "Attention Is All You Need," has transformed the field of natural language processing (NLP) by enabling models to capture how words are related in context. BERT [6] and its variants further extended this by learning bidirectional representations, which underpin current sentence embeddings.

Benefits of Transformer Embeddings:

- Dense Representations: Usually, vectors have 384 to 768 dimensions, while TF-IDF vectors have thousands.
- Semantic understanding: Gets the meaning, not simply the words.
- Context awareness: Knows what words mean in the context they are used in
- Transfer learning: learns from vast amounts of data, gaining knowledge about the world.

5.5 Recommendations and Rankings

The final step in the pipeline calculates similarity scores, ranks the findings, and generates helpful suggestions.

5.5.1 Finding Similarities

The cosine similarity between the resume embedding and each job description embedding in the database is calculated. Cosine similarity tells you how far apart two vectors are in the embedding space. The values can go from -1 (opposite) to 1 (same). For semantic embeddings, a higher cosine similarity means that the meanings are more closely aligned.

5.5.2 Scoring in More Than One Dimension

The total score for the match takes into account several factors:

- Skills-Requirements similarity: How closely do the candidate's skills match what the employer needs?
- Similarity between experience and responsibilities: Does the candidate's experience match the job's duties?

5.5.3 Suggestion Output

We filter jobs by their similarity score and then recommend the top-N jobs. There are the following things in each suggestion:

- Matching Skills: The skills the candidate has that are needed for the job
- Missing Skills (Gap Analysis): Skills that the candidate needs but doesn't have.
- Similarity Confidence Score: A number that shows how strong a match is.
- Profile Match Strength: A qualitative evaluation of how well the two profiles fit together.

This detailed output provides job seekers with clear steps to take and recruiters with a quick way to assess whether a prospect is a good fit.

6. Model Used

Our solution incorporates various models, each designed for a distinct function within the recommendation pipeline.

Model	Type	Purpose	Strength
Qwen	LLM	Skill Extraction	Context & Reasoning
GLiNER	NER	Entity Detection	Precision & Speed
Sentence Transformers	Embedding	Vectorization	Semantic Similarity

6.1 Qwen (Extensive Linguistic Model)

Qwen is an advanced large language model developed by Alibaba [7]. In our system, Qwen performs the intricate function of extracting abilities, responsibilities, and experience from unstructured text.

Core functionalities of our system include:

- Extraction of skills, responsibilities, and experience from resumes and job descriptions.
- Comprehension of context and identification of implicit skills not overtly mentioned.
- Robust semantic extraction via reasoning.
- Management of ambiguous or inadequately formatted input documents.

6.2 GLiNER (Named Entity Recognition Model)

GLiNER (Generalist and Lightweight Model for Named Entity Recognition) is an LLM-based model that enhances extraction by improving entity identification accuracy [8].

Essential functionalities of our system include:

- Accurate detection of explicit skills and technical things.
- Token-level classification with little hallucination.
- Accelerated processing compared to comprehensive LLM inference.
- Incorporation of NER precision alongside contextual comprehension.

6.3 Sentence Transformers (Embedding Model)

Sentence Transformers is a framework for generating dense vector representations of sentences and documents [6], [9]. These embeddings underlie our similarity calculation.

Essential functionalities within our system:

- Transforms complete resumes and job descriptions into compact semantic vectors.
- Encapsulates meaning, role similarity, and contextual subtleties.
- Facilitates efficient computation of cosine similarity for matching.
- Pretrained on varied corpora for extensive domain coverage.

7. Design Rationale: Unsupervised Learning

An essential design choice in our system is to use unsupervised learning rather than supervised methods. This section clarifies the reasoning for this decision.

7.1 No Labelled Datasets

The main problem with supervised job matching is that there isn't enough good training data:

- No public datasets: There isn't a publicly available dataset that has "Resume + Job → Good/Bad match" labels.
- Not possible to label anything by hand: Making these kinds of labelling is costly, takes a lot of work, and is essentially subjective.
- Domain specificity: Each domain and industry necessitates distinct labelled datasets.
- Ground truth ambiguity: What makes a "good match" is typically different for each person and situation.

7.2 Pretrained Models Encode Knowledge

Modern LLMs and embedding models are trained on large-scale datasets containing billions of words. This pretraining aggregates substantial information about the world, including how abilities, job terms, and professional situations are interrelated. Zero-shot semantic similarity comparison of embeddings, without task-specific training, is effective for job matching because the model weights already encode these links.

7.3 Natural Fit for Similarity-Based Matching

Similarity-based matching is not supervised by nature. We are not trying to determine whether resume-job combinations are "good" or "bad." Instead, we are figuring out how similar they are. We do this task without supervision by measuring distances in embedding space without using labelled instances.

7.4 Maintainability and Adaptability

Unsupervised methods have a lot of real-world benefits:

- No need to retrain: You can add new skills without having to retrain the system.
- Updates are easy: You may easily add new entries to the skill ontology.
- The same system works in different sectors without any changes.
- You don't have to keep collecting and labelling new training data.

8. Implementation Details

This section outlines the technical specifications regarding the execution of our Job Recommendation System.

8.1 Stack

The system employs the following kinds of technologies:

Component	Technology
Programming Language	Python
LLM Framework	Qwen (Alibaba), Transformers library
NER Model	GLiNER
Embeddings	Sentence Transformers (all-MiniLM, all-mpnet)
PDF Processing	PyPDF2, pdfplumber
NLP Libraries	spaCy, NLTK
Data Processing	Pandas, NumPy, scikit-learn
UI Framework	Streamlit

8.2 System Requirements

For the best performance, the system needs the following minimal requirements:

- **CPU:** Multi-core processor (4+ cores recommended)
- **RAM:** 16GB minimum, 32GB recommended for LLM inference
- **GPU:** NVIDIA GPU with CUDA support (optional but recommended)
- **Storage:** 10GB+ for models and data

8.3 Code Structure

The project is divided into distinct components, each responsible for a specific task, such as document processing, skill extraction, embedding generation, and recommendation generation.

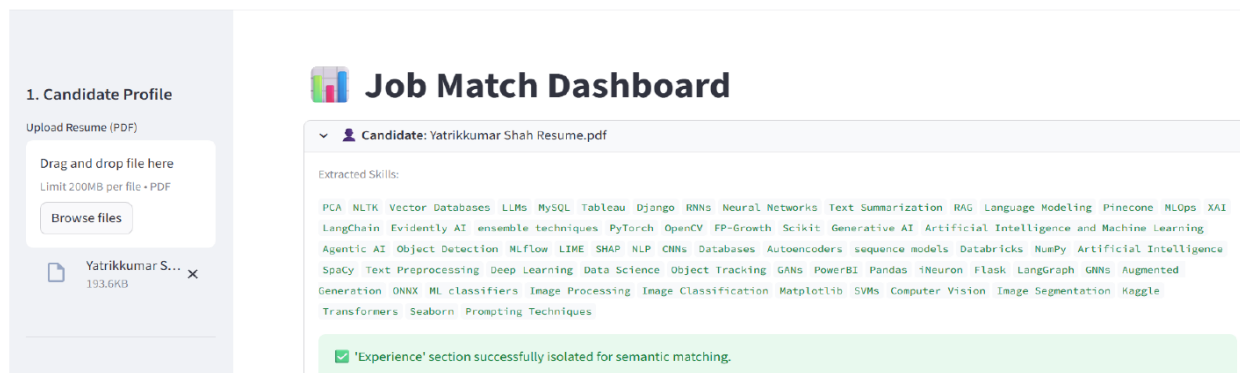
You can find the whole source code at www.github.com/Yatrik07/Job-Recommender-System.

9. Application Demo and Results

The system has been developed as a web application using Streamlit[10], making it easy for both job seekers and recruiters to use.

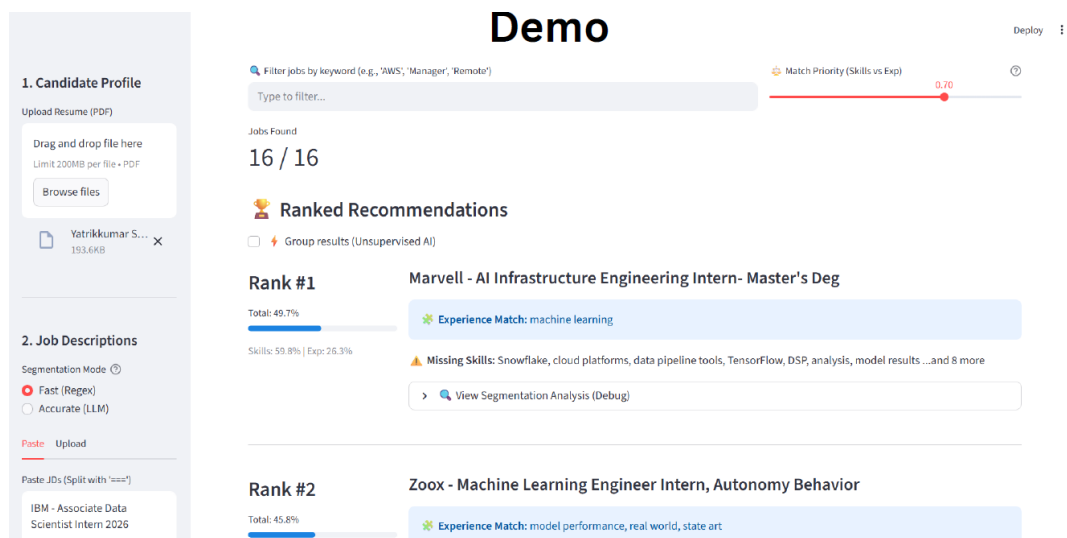
9.1 User Interface

Users can upload their resumes in PDF format through the app's simple, clean layout. After that, the algorithm reviews the paper, identifies talents, and presents suggested jobs along with match scores and skill evaluations.



9.2 Sample Output

When a user uploads their resume, the system shows them suggestions for jobs, firms, match scores, skills that are a good fit for the position (in green), and skills that the user needs to work on (in red).



10. Future Scope

The current system does a good job of suggesting jobs, but there are plans to improve it in the future.

10.1 Model Improvement

Fine-tuning for Domain Specificity: Specifically fine-tune transformer and LLM models for extracting skills and classifying domains. This might improve accuracy in industries such as healthcare, finance, and engineering.

Advanced Reranking: Use larger models, such as Qwen-Large or LLaMA, to rerank the top recommendations to make them more semantically relevant. A two-step process (a quick initial rating followed by detailed reranking) might provide both speed and quality.

10.2 User Feedback Integration

A significant improvement would be to add a user feedback loop so that the algorithm can learn from:

- Jobs the user applied for (good sign)
- Jobs the user liked or saved (interest sign)
- Jobs the user skipped (bad sign)
- The truth about interview results and hiring choices

10.3 Enhanced User Interface

Use Streamlit or React to develop an interactive UI dashboard with elements such as visual skill graphs, career path suggestions, comparison views, and personalised insights based on the user's past actions.

10.4 Advanced Filtering and Business Logic

Add weighted scoring and business filters, such as:

- Salary range: Sort and rank jobs by how well they pay.
- Location preferences: Support filtering by location, remote work, and hybrid work.
- Experience level: Match jobs to the candidate's level of experience.
- Job category: Filter by industry and function.
- Company ratings: Include ratings and evaluations from employers.

Conclusion

This project has successfully developed a Job Recommendation System that overcomes the fundamental limitations of conventional keyword-based job matching. Utilising contemporary NLP methodologies, Large Language Models (Qwen), Named Entity Recognition (GLiNER), and Sentence Transformer embeddings, we have developed a system that comprehensively comprehends the semantic significance of talents, experiences, and job prerequisites. The hybrid extraction method provides comprehensive coverage of skills, while the ontology-driven normalisation effectively addresses variants, synonyms, and abbreviations. The unsupervised learning methodology eliminates the necessity for costly labelled datasets while preserving high accuracy through zero-shot semantic similarity, rendering the system practical, scalable, and easy to maintain.

The method delivers actionable recommendations derived from skills, experience, contextual similarity, and rating confidence, thereby providing substantial benefits for both job seekers and recruiters relative to unreliable keyword searches. Job searchers obtain pertinent job matching accompanied by skill gap assessments for professional advancement, whereas recruiters can effectively pinpoint competent individuals without constraints imposed by terminological discrepancies. Future additions, such as incorporating user feedback, sophisticated reranking algorithms, and business filters for salary, location, and experience level, will enhance the quality of recommendations. As AI and NLP technologies progress, semantic-based matching systems such as ours will assume a more significant role in connecting talent with opportunity.

References

- [1] “Job Descriptions 2025 – Tech & Non-Tech Roles.” Accessed: Dec. 11, 2025. [Online]. Available: <https://www.kaggle.com/datasets/adityarajsrv/job-descriptions-2025-tech-and-non-tech-roles>
- [2] “spaCy · Industrial-strength Natural Language Processing in Python.” Accessed: Dec. 11, 2025. [Online]. Available: <https://spacy.io/>
- [3] T. Mikolov, I. Sutskever, K. Chen, G. Corrado, and J. Dean, “Distributed Representations of Words and Phrases and their Compositionality,” Oct. 2013, [Online]. Available: <http://arxiv.org/abs/1310.4546>
- [4] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” May 2019, [Online]. Available: <http://arxiv.org/abs/1810.04805>
- [5] A. Vaswani *et al.*, “Attention Is All You Need.”
- [6] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” Aug. 2019, [Online]. Available: <http://arxiv.org/abs/1908.10084>
- [7] J. Bai *et al.*, “Qwen Technical Report,” Sep. 2023, [Online]. Available: <http://arxiv.org/abs/2309.16609>
- [8] U. Zaratiana, N. Tomeh, P. Holat, and T. Charnois, “GLiNER: Generalist Model for Named Entity Recognition using Bidirectional Transformer,” Nov. 2023, [Online]. Available: <http://arxiv.org/abs/2311.08526>
- [9] “SentenceTransformers Documentation — Sentence Transformers documentation.” Accessed: Dec. 11, 2025. [Online]. Available: <https://sbert.net/>
- [10] “Streamlit documentation.” Accessed: Dec. 11, 2025. [Online]. Available: <https://docs.streamlit.io/>